

Control Statements and Program Development

3



Objectives

In this chapter, you'll:

- Decide whether to execute actions with the statements `if`, `if...else` and `if...elif...else`.
- Execute statements repeatedly with `while` and `for`.
- Shorten assignment expressions with augmented assignments.
- Use the `for` statement and the built-in `range` function to repeat actions for a sequence of values.
- Perform sentinel-controlled repetition with `while`.
- Learn problem-solving skills: understanding problem requirements, dividing problems into smaller pieces, developing algorithms to solve problems and implementing those algorithms in code.
- Develop algorithms through the process of top-down, stepwise refinement.
- Create compound conditions with the Boolean operators `and`, `or` and `not`.
- Stop looping with `break`.
- Force the next iteration of a loop with `continue`.
- Use some functional-style programming features to write scripts that are more concise, clearer, easier to debug and easier to parallelize.

3.1 Introduction	3.11 Program Development: Sentinel-Controlled Repetition
3.2 Algorithms	3.12 Program Development: Nested Control Statements
3.3 Pseudocode	3.13 Built-In Function <code>range</code> : A Deeper Look
3.4 Control Statements	3.14 Using Type <code>Decimal</code> for Monetary Amounts
3.5 <code>if</code> Statement	3.15 <code>break</code> and <code>continue</code> Statements
3.6 <code>if...else</code> and <code>if...elif...else</code> Statements	3.16 Boolean Operators <code>and</code> , <code>or</code> and <code>not</code>
3.7 <code>while</code> Statement	3.17 Intro to Data Science: Measures of Central Tendency—Mean, Median and Mode
3.8 <code>for</code> Statement	3.18 Wrap-Up Exercises
3.8.1 Iterables, Lists and Iterators	
3.8.2 Built-In <code>range</code> Function	
3.9 Augmented Assignments	
3.10 Program Development: Sequence-Controlled Repetition	
3.10.1 Requirements Statement	
3.10.2 Pseudocode for the Algorithm	
3.10.3 Coding the Algorithm in Python	
3.10.4 Introduction to Formatted Strings	

3.1 Introduction

Before writing a program to solve a particular problem, you must understand the problem and have a carefully planned approach to solving it. You must also understand Python's building blocks and use proven program-construction principles.

3.2 Algorithms

You can solve any computing problem by executing a series of actions in a specific order. An **algorithm** is a *procedure* for solving a problem in terms of:

1. the **actions** to execute, and
2. the **order** in which these actions execute.

Correctly specifying the order in which the actions execute is essential. Consider the “rise-and-shine algorithm” that an executive follows for getting out of bed and going to work: (1) Get out of bed; (2) take off pajamas; (3) take a shower; (4) get dressed; (5) eat breakfast; (6) carpool to work. This routine gets the executive to work well prepared to make critical decisions. Suppose the executive performs these steps in a different order: (1) Get out of bed; (2) take off pajamas; (3) get dressed; (4) take a shower; (5) eat breakfast; (6) carpool to work. Now, our executive shows up for work soaking wet. **Program control** specifies the order in which statements (actions) execute in a program. This chapter investigates program control using Python's **control statements**.



Self Check

I (*Fill-In*) A(n) _____ is a procedure for solving a problem. It specifies the _____ to execute and the _____ in which they execute.

Answer: algorithm, actions, order.

3.3 Pseudocode

Pseudocode is an informal English-like language for “thinking out” algorithms. You write text that describes what your program should do. You then convert the pseudocode to Python by replacing pseudocode statements with their Python equivalents.

Addition-Program Pseudocode

The following pseudocode algorithm prompts the user to enter two integers, inputs them from the user at the keyboard, adds them, then stores and displays their sum:

Prompt the user to enter the first integer

Input the first integer

Prompt the user to enter the second integer

Input the second integer

Add first integer and second integer, store their sum

Display the numbers and their sum

This is the complete pseudocode algorithm. Later in the chapter, we’ll show a simple process for creating a pseudocode algorithm from a *requirements statement*. The English pseudocode statements specify the actions you wish to perform and the order in which you wish to perform them.



Self Check

1 (*True/False*) Pseudocode is a simple programming language.

Answer: False. Pseudocode is not a programming language. It’s an artificial and informal language that helps you develop algorithms.

2 (*IPython Session*) Write Python statements that perform the tasks described by this section’s pseudocode. Enter the integers 10 and 5.

Answer:

```
In [1]: number1 = int(input('Enter first integer: '))
Enter first integer: 10
```

```
In [2]: number2 = int(input('Enter second integer: '))
Enter second integer: 5
```

```
In [3]: total = number1 + number2
```

```
In [4]: print('The sum of', number1, 'and', number2, 'is', total)
The sum of 10 and 5 is 15
```

3.4 Control Statements

Usually, statements in a program execute in the order in which they’re written. This is called *sequential execution*. Various Python statements enable you to specify that the next statement to execute may be *other than* the next one *in sequence*. This is called *transfer of control* and is achieved with Python *control statements*.

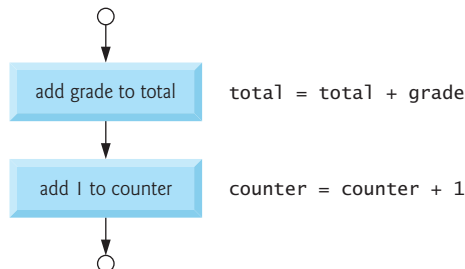
Forms of Control

In the 1960s, extensive use of control transfers was causing difficulty in software development. Blame was pointed at the `goto` statement. This statement allowed you to transfer control to one of many possible destinations in a program. Bohm and Jacopini's research¹ demonstrated that programs could be written without `goto` statements. The notion of *structured programming* became almost synonymous with “goto elimination.” Python does not have a `goto` statement. Structured programs are clearer, easier to debug and change, and more likely to be bug-free.

Bohm and Jacopini demonstrated that all programs could be written using three forms of control—namely, **sequential execution**, the **selection statement** and the **repetition statement**. Sequential execution is simple. Python statements execute one after the other “in sequence,” unless directed otherwise.

Flowcharts

A **flowchart** is a *graphical* representation of an algorithm or a part of one. You draw flowcharts using *rectangles*, *diamonds*, *rounded rectangles* and *small circles* that you connect by *arrows* called **flowlines**. Like pseudocode, flowcharts are useful for developing and representing algorithms. They clearly show how forms of control operate. Consider the following flowchart segment, which shows *sequential execution*:



We use the **rectangle (or action) symbol** to indicate any *action*, such as a calculation or an input/output operation. The flowlines show the *order* in which the actions execute. First, the grade is added to the total, then 1 is added to the counter. We show the Python code next to each action symbol for comparison purposes. This code is not part of the flowchart.

In a flowchart for a *complete* algorithm, the first symbol is a **rounded rectangle** containing the word “Begin.” The last symbol is a rounded rectangle containing the word “End.” In a flowchart for only a *part* of an algorithm, we omit the rounded rectangles, instead using small circles called **connector symbols**. The most important symbol is the **decision (or diamond) symbol**, which indicates that a *decision* is to be made, such as in an `if` statement. We begin using decision symbols in the next section.

Selection Statements

Python provides three types of selection statements that execute code based on a *condition*—an expression that evaluates to either `True` or `False`:

1. Bohm, C., and G. Jacopini, “Flow Diagrams, Turing Machines, and Languages with Only Two Formation Rules,” *Communications of the ACM*, Vol. 9, No. 5, May 1966, pp. 336–371.

- The `if` statement performs an action if a condition is `True` or skips the action if the condition is `False`.
- The **`if...else` statement** performs an action if a condition is `True` or performs a *different* action if the condition is `False`.
- The **`if...elif...else` statement** performs one of many different actions, depending on the truth or falsity of *several* conditions.

Anywhere a single action can be placed, a group of actions can be placed.

The `if` statement is called a **single-selection statement** because it selects or ignores a *single* action (or group of actions). The `if...else` statement is called a **double-selection statement** because it selects between *two different* actions (or groups of actions). The `if...elif...else` statement is called a **multiple-selection statement** because it selects one of many different actions (or groups of actions).

Repetition Statements

Python provides two repetition statements—`while` and `for`:

- The `while` statement repeats an action (or a group of actions) as long as a condition remains `True`.
- The `for` statement repeats an action (or a group of actions) for every item in a sequence of items.

Keywords

The words `if`, `elif`, `else`, `while`, `for`, `True` and `False` are keywords that Python reserves to implement its features, such as control statements. Using a keyword as an identifier such as a variable name is a syntax error. The following table lists Python's keywords.

Python keywords

<code>and</code>	<code>as</code>	<code>assert</code>	<code>async</code>	<code>await</code>	<code>break</code>	<code>class</code>
<code>continue</code>	<code>def</code>	<code>del</code>	<code>elif</code>	<code>else</code>	<code>except</code>	<code>False</code>
<code>finally</code>	<code>for</code>	<code>from</code>	<code>global</code>	<code>if</code>	<code>import</code>	<code>in</code>
<code>is</code>	<code>lambda</code>	<code>None</code>	<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>
<code>raise</code>	<code>return</code>	<code>True</code>	<code>try</code>	<code>while</code>	<code>with</code>	<code>yield</code>

Control Statements Summary

You form each Python program by combining as many control statements of each type as you need for the algorithm the program implements. With **Single-entry/single-exit** (one way in/one way out) **control statements**, the exit point of one connects to the entry point of the next. This is similar to the way a child stacks building blocks—hence, the term **control-statement stacking**. **Control-statement nesting** also connects control statements—we'll see how later in the chapter.

You can construct any Python program from only six different forms of control (sequential execution, and the `if`, `if...else`, `if...elif...else`, `while` and `for` statements). You combine these in only two ways (control-statement stacking and control-statement nesting). This is the essence of simplicity.

✓ Self Check

1 (Fill-In) You can write all programs using three forms of control—_____, _____ and _____.

Answer: sequential execution, selection statements, repetition statements.

2 (Fill-In) A(n) _____ is a graphical representation of an algorithm.

Answer: flowchart.

3.5 if Statement

Suppose that a passing grade on an examination is 60. The pseudocode

*If student's grade is greater than or equal to 60
Display 'Passed'*

determines whether the condition “student’s grade is greater than or equal to 60” is true or false. If the condition is true, 'Passed' is displayed. Then, the next pseudocode statement in order is “performed.” (Remember that pseudocode is not a real programming language.) If the condition is false, nothing is displayed, and the next pseudocode statement is “performed.” The pseudocode’s second line is indented. Python code requires indentation. Here it emphasizes that 'Passed' is displayed *only* if the condition is true.

Let’s assign 85 to the variable grade, then show and execute the Python `if` statement for the pseudocode:

```
In [1]: grade = 85

In [2]: if grade >= 60:
...:     print('Passed')
...:
Passed
```

The `if` statement closely resembles the pseudocode. The condition `grade >= 60` is `True`, so the indented `print` statement displays 'Passed'.

Suite Indentation

Indenting a suite is required; otherwise, an `IndentationError` syntax error occurs:

```
In [3]: if grade >= 60:
...:     print('Passed') # statement is not indented properly
File "<ipython-input-3-f42783904220>", line 2
    print('Passed') # statement is not indented properly
    ^
IndentationError: expected an indented block
```

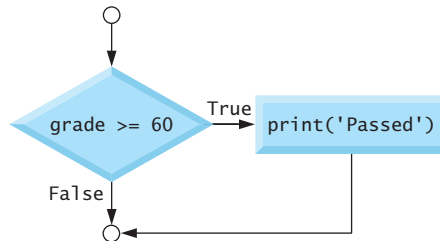
An `IndentationError` also occurs if you have more than one statement in a suite and those statements do not have the *same* indentation:

```
In [4]: if grade >= 60:
...:     print('Passed') # indented 4 spaces
...:     print('Good job!') # incorrectly indented only two spaces
File <ipython-input-4-8c0d75c127bf>, line 3
    print('Good job!') # incorrectly indented only two spaces
    ^
IndentationError: unindent does not match any outer indentation level
```

Sometimes error messages may not be clear. The fact that Python calls attention to the line is usually enough for you to figure out what's wrong. Apply indentation conventions uniformly throughout your code. Programs that are not uniformly indented are hard to read.

if Statement Flowchart

The flowchart for the if statement in snippet [2] is:



The decision (diamond) symbol contains a condition that can be either True or False. The diamond has two flowlines emerging from it:

- One indicates the direction to follow when the condition in the symbol is True. This points to the action (or group of actions) that should execute.
- The other indicates the direction to follow when the condition is False. This skips the action (or group of actions).

Every Expression Can Be Interpreted as Either True or False

You can base decisions on *any* expression. A nonzero value is True. Zero is False:

```
In [5]: if 1:
...     print('Nonzero values are true, so this will print')
...
Nonzero values are true, so this will print

In [6]: if 0:
...     print('Zero is false, so this will not print')

In [7]:
```

Strings containing characters are True and empty strings ('', ''', or ''''') are False.

An Additional Note on Confusing == and =

Using the equality operator == instead of the assignment symbol = in an assignment statement can lead to subtle problems. For example, in this session, snippet [1] defined grade with the assignment:

```
grade = 85
```

If instead we accidentally wrote:

```
grade == 85
```

then grade would be undefined and we'd get a NameError.

If grade had been defined before the preceding statement, then grade == 85 would evaluate to True or False, depending on grade's value, and not perform the intended assignment. This is a logic error.

✓ Self Check

1 (True/False) If you indent a suite's statements, you will not get an `IndentationError`.
Answer: False. All the statements in a suite must have the *same* indentation. Otherwise, an `IndentationError` occurs.

2 (IPython Session) Redo this section's snippets [1] and [2], then change `grade` to 55 and repeat the `if` statement to show that its suite does not execute. The next section shows how to recall and re-execute earlier snippets to avoid having to re-enter the code.

Answer:

```
In [1]: grade = 85

In [2]: if grade >= 60:
...:     print('Passed')
...:
Passed

In [3]: grade = 55

In [4]: if grade >= 60:
...:     print('Passed')
...:

In [5]:
```

3.6 if...else and if...elif...else Statements

The `if...else` statement performs different suites, based on whether a condition is `True` or `False`. The pseudocode below displays 'Passed' if the student's grade is greater than or equal to 60; otherwise, it displays 'Failed':

```
If student's grade is greater than or equal to 60
    Display 'Passed'
Else
    Display 'Failed'
```

In either case, the next pseudocode statement in sequence after the entire *If...Else* is “performed.” We indent both the *If* and *Else* suites, and by the same amount. Let's create and **initialize** (that is, give a starting value to) the variable `grade`, then show and execute the Python `if...else` statement for the preceding pseudocode:

```
In [1]: grade = 85

In [2]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:
Passed
```

The condition above is `True`, so the `if` suite displays 'Passed'. Note that when you press *Enter* after typing `print('Passed')`, IPython indents the next line four spaces. You must delete those four spaces so that the `else:` suite correctly aligns under the `i` in `if`.

The following code assigns 57 to the variable `grade`, then shows the `if...else` statement again to demonstrate that only the `else` suite executes when the condition is `False`:

```
In [3]: grade = 57

In [4]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:
Failed
```

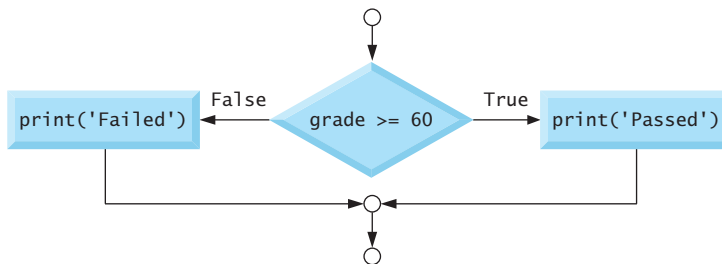
The up and down arrow keys navigate backwards and forwards through the current interactive session's snippets. Pressing *Enter* re-executes the snippet that's displayed. Let's set `grade` to 99, press the up arrow key twice to recall the code from snippet [4], then press *Enter* to re-execute that code as snippet [6]. Every recalled snippet that you execute gets a new ID:

```
In [5]: grade = 99

In [6]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:
Passed
```

if...else Statement Flowchart

The flowchart below shows the preceding `if...else` statement's flow of control:



Conditional Expressions

Sometimes the suites in an `if...else` statement assign different values to a variable, based on a condition, as in:

```
In [7]: grade = 87

In [8]: if grade >= 60:
...:     result = 'Passed'
...: else:
...:     result = 'Failed'
...:
```

We can then print or evaluate that variable:

```
In [9]: result
Out[9]: 'Passed'
```

You can write statements like snippet [8] using a concise **conditional expression**:

```
In [10]: result = ('Passed' if grade >= 60 else 'Failed')
```

```
In [11]: result
Out[11]: 'Passed'
```

The parentheses are not required, but they make it clear that the statement assigns the conditional expression's value to `result`. First, Python evaluates the condition `grade >= 60`:

- If it's True, snippet [10] assigns to `result` the value of the expression to the *left* of `if`, namely 'Passed'. The `else` part does not execute.
- If it's False, snippet [10] assigns to `result` the value of the expression to the *right* of `else`, namely 'Failed'.

In interactive mode, you also can evaluate the conditional expression directly, as in:

```
In [12]: 'Passed' if grade >= 60 else 'Failed'
Out[12]: 'Passed'
```

Multiple Statements in a Suite

The following code shows two statements in the `else` suite of an `if...else` statement:

```
In [13]: grade = 49

In [14]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:     print('You must take this course again')
...:
Failed
You must take this course again
```

In this case, `grade` is less than 60, so *both* statements in the `else`'s suite execute. If you do not indent the second `print`, then it's not in the `else`'s suite. So, that statement *always* executes, creating strange incorrect output:

```
In [15]: grade = 100

In [16]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...: print('You must take this course again')
...:
Passed
You must take this course again
```

if...elif...else Statement

You can test for many cases using the **if...elif...else statement**. The following pseudocode displays “A” for grades greater than or equal to 90, “B” for grades in the range 80–89, “C” for grades 70–79, “D” for grades 60–69 and “F” for all other grades:

```
If student's grade is greater than or equal to 90
    Display "A"
Else If student's grade is greater than or equal to 80
    Display "B"
Else If student's grade is greater than or equal to 70
    Display "C"
Else If student's grade is greater than or equal to 60
    Display "D"
Else
    Display "F"
```

Only the action for the first True condition executes. Let's show and execute the Python code for the preceding pseudocode. The pseudocode *Else If* is written with the keyword `elif`. Snippet [18] displays C, because grade is 77:

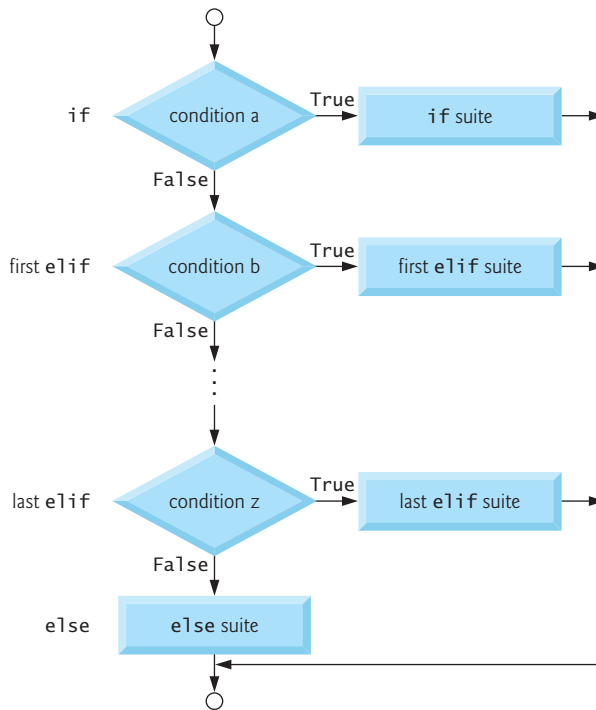
```
In [17]: grade = 77

In [18]: if grade >= 90:
...:     print('A')
...: elif grade >= 80:
...:     print('B')
...: elif grade >= 70:
...:     print('C')
...: elif grade >= 60:
...:     print('D')
...: else:
...:     print('F')
...:
C
```

The first condition—`grade >= 90`—is False, so `print('A')` is skipped. The second condition—`grade >= 80`—also is False, so `print('B')` is skipped. The third condition—`grade >= 70`—is True, so `print('C')` executes. Then all the remaining code in the `if...elif...else` statement is skipped. An `if...elif...else` is faster than separate `if` statements, because condition testing stops as soon as a condition is True.

if...elif...else Statement Flowchart

The following flowchart shows the general flow through an `if...elif...else` statement. It shows that, after any suite executes, control immediately exits the statement. The words to the left are not part of the flowchart. We added them to show how the flowchart corresponds to the equivalent Python code.



else Is Optional

The `else` in the `if...elif...else` statement is optional. Including it enables you to handle values that do not satisfy *any* of the conditions. When an `if...elif` statement without an `else` tests a value that does not make any of its conditions `True`, the program does not execute any of the statement's suites. The next statement in sequence after the `if...elif` statement executes. If you specify the `else`, you must place it after the last `elif`; otherwise, a `SyntaxError` occurs.

Logic Errors

The incorrectly indented code segment in snippet [16] is an example of a **nonfatal logic error**. The code executes, but it produces incorrect results. For a **fatal logic error** in a script, an exception occurs (such as a `ZeroDivisionError` from an attempt to divide by 0), so Python displays a traceback, then terminates the script. A fatal error in interactive mode terminates only the current snippet. Then IPython waits for your next input.



Self Check

1 (*True/False*) A fatal logic error causes a script to produce incorrect results, then continue executing.

Answer: False. A fatal logic error causes a script to terminate.

2 (*IPython Session*) Show that a `SyntaxError` occurs if an `if...elif` statement specifies an `else` before the last `elif`.

Answer:

```
In [1]: grade = 80

In [2]: if grade >= 90:
...:     print('A')
...: else:
...:     print('Not A or B')
...: elif grade >= 80:
File "<ipython-input-2-033bcba40157>", line 5
    elif grade >= 80:
        ^
SyntaxError: invalid syntax
```

3.7 while Statement

The **while statement** allows you to *repeat* one or more actions while a condition remains True. Such a statement often is called a **loop**.

The following pseudocode specifies what happens when you go shopping:

*While there are more items on my shopping list
Buy next item and cross it off my list*

If the condition “there are more items on my shopping list” is *true*, you perform the action “Buy next item and cross it off my list.” You *repeat* this action while the condition remains *true*. You stop repeating this action when the condition becomes *false*—that is, when you’ve crossed all items off your shopping list.

Let’s use a `while` statement to find the first power of 3 larger than 50:

```
In [1]: product = 3

In [2]: while product <= 50:
...:     product = product * 3
...:

In [3]: product
Out[3]: 81
```

First, we create `product` and initialize it to 3. Then the `while` statement executes as follows:

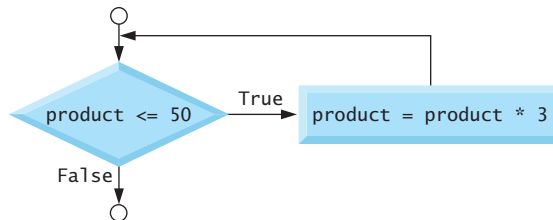
1. Python tests the condition `product <= 50`, which is `True` because `product` is 3. The statement in the suite multiplies `product` by 3 and assigns the result (9) to `product`. One *iteration of the loop* is now complete.
2. Python again tests the condition, which is `True` because `product` is now 9. The suite’s statement sets `product` to 27, completing the second iteration of the loop.
3. Python again tests the condition, which is `True` because `product` is now 27. The suite’s statement sets `product` to 81, completing the third iteration of the loop.
4. Python again tests the condition, which is finally `False` because `product` is now 81. The repetition now terminates.

Snippet [3] evaluates `product` to see its value, 81, which is the first power of 3 larger than 50. If this `while` statement were part of a larger script, execution would continue with the next statement in sequence after the `while`.

Something in the `while` statement's suite must change `product`'s value, so the condition eventually becomes `False`. Otherwise, a logic error called an **infinite loop** occurs. Such an error prevents the `while` statement from ever terminating—the program appears to “hang.” In applications executed from a Terminal, Command Prompt or shell, type `Ctrl + c` or `control + c` (depending on your keyboard) to terminate an infinite loop. IDEs typically have a toolbar button or menu option for stopping a program's execution.

while Statement Flowchart

The following flowchart shows the preceding `while` statement's flow of control:



Follow the flowlines to experience the repetition. The flowline from the rectangle “closes the loop” by flowing back into the condition `product <= 50` that's tested during each iteration. When that condition becomes `False`, the `while` statement exits and control proceeds to the next statement in sequence.

✓ Self Check

1 (*True/False*) A `while` statement performs its suite while some condition remains `True`.
Answer: `True`.

2 (*IPython Session*) Write statements to determine the first power of 7 greater than 1000.
Answer:

```

In [1]: product = 7

In [2]: while product <= 1000:
...:     product = product * 7
...:

In [3]: product
Out[3]: 2401

```

3.8 for Statement

Like the `while` statement, the **for statement** allows you to *repeat* an action or several actions. The `for` statement performs its action(s) for each item in a **sequence** of items. For example, a string is a sequence of individual characters. Let's display 'Programming' with its characters separated by two spaces:

```

In [1]: for character in 'Programming':
...:     print(character, end='  ')
...:

P  r  o  g  r  a  m  m  i  n  g

```

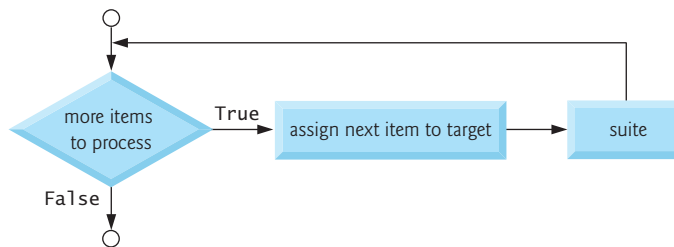
The for statement executes as follows:

- Upon entering the statement, it assigns the 'P' in 'Programming' to the **target** variable between keywords `for` and `in`—in this case, `character`.
- Next, the statement in the suite executes, displaying `character`'s value followed by two spaces—we'll say more about this momentarily.
- After executing the suite, Python assigns to `character` the next item in the sequence (that is, the 'r' in 'Programming'), then executes the suite again.
- This continues while there are more items in the sequence to process. In this case, the statement terminates after displaying the letter 'g', followed by two spaces.

Using the target in the suite, as we did here to display its value, is common but not required.

for Statement Flowchart

The for statement's flowchart is similar to that of the `while` statement:



First, Python determines whether there are more items to process. If so, the `for` statement assigns the next item to the target, then performs the suite's action(s).

Function `print`'s `end` Keyword Argument

The built-in function `print` displays its argument(s), then moves the cursor to the next line. You can change this behavior with the argument **`end`**, as in

```
print(character, end='  ')
```

We used two spaces (' '), so each call to `print` displays `character`'s value followed by two spaces. So, all the characters display horizontally on the *same* line. Python calls **`end`** a **keyword argument**, but `end` is not a Python keyword. The `end` keyword argument is *optional*. If you do not include it, `print` uses a newline ('\n') by default. The *Style Guide for Python Code* recommends placing no spaces around a keyword argument's `=`. Keyword arguments are sometimes called named arguments.

Function `print`'s `sep` Keyword Argument

You can use the keyword argument **`sep`** (short for separator) to specify the string that appears *between* the items that `print` displays. When you do not specify this argument, `print` uses a space character by default. Let's display three numbers, each separated from the next by a comma and a space, rather than just a space:

```
In [2]: print(10, 20, 30, sep=', ')
10, 20, 30
```

To remove the spaces, use an **empty string** with no characters between its quotes.

3.8.1 Iterables, Lists and Iterators

The sequence to the right of the `for` statement's `in` keyword must be an **iterable**. An iterable is an object from which the `for` statement can take one item at a time until no more items remain. Python has other iterable sequence types besides strings. One of the most common is a **list**, which is a comma-separated collection of items enclosed in square brackets (`[` and `]`). The following code totals five integers in a list:

```
In [3]: total = 0

In [4]: for number in [2, -3, 0, 17, 9]:
...:     total = total + number
...:

In [5]: total
Out[5]: 25
```

Each sequence has an **iterator**. The `for` statement uses the iterator “behind the scenes” to get each consecutive item until there are no more to process. The iterator is like a bookmark—it always knows where it is in the sequence, so it can return the next item when it's called upon to do so.

For each `number` in the list, the suite adds the `number` to the `total`. When there are no more items to process, `total` contains the sum (25) of the list's items. We cover lists in detail in the “Sequences: Lists and Tuples” chapter. There, you'll see that the order of the items in a list matters and that a list's items are **mutable** (that is, modifiable).

3.8.2 Built-In range Function

Let's use a `for` statement and the built-in **range function** to iterate precisely 10 times, displaying the values from 0 through 9:

```
In [6]: for counter in range(10):
...:     print(counter, end=' ')
...:
0 1 2 3 4 5 6 7 8 9
```

The function call `range(10)` creates an iterable object that represents a sequence of consecutive integer values starting from 0 and continuing up to, but *not* including, the argument value (10). In this case, the sequence is 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. The `for` statement exits when it finishes processing the last integer that `range` produces. Iterators and iterable objects are two of Python's *functional-style programming* features. We'll introduce more of these throughout the book.

Off-By-One Errors

A logic error known as an **off-by-one error** occurs when you assume that `range`'s argument value is included in the generated sequence. For example, if you provide 9 as `range`'s argument when trying to produce the sequence 0 through 9, `range` generates only 0 through 8.



Self Check

1 (Fill-In) Function _____ generates a sequence of integers.

Answer: `range`.

2 (*IPython Session*) Use the range function and a for statement to calculate the total of the integers from 0 through 1,000,000.

Answer:

```
In [1]: total = 0

In [2]: for number in range(1000001):
...:     total = total + number
...:

In [3]: total
Out[3]: 500000500000
```

3.9 Augmented Assignments

Augmented assignments abbreviate assignment expressions in which the same variable name appears on the left and right of the assignment's `=`, as `total` does in:

```
for number in [1, 2, 3, 4, 5]:
    total = total + number
```

Snippet [2] reimplements this using an **addition augmented assignment (`+=`) statement**:

```
In [1]: total = 0

In [2]: for number in [1, 2, 3, 4, 5]:
...:     total += number # add number to total and store in number
...:

In [3]: total
Out[3]: 15
```

The `+=` expression in snippet [2] first adds `number`'s value to the current `total`, then stores the new value in `total`. The table below shows sample augmented assignments:

Augmented assignment	Sample expression	Explanation	Assigns
<i>Assume: c = 3, d = 5, e = 4, f = 2, g = 9, h = 12</i>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 to c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 to d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 to e
<code>**=</code>	<code>f **= 3</code>	<code>f = f ** 3</code>	8 to f
<code>/=</code>	<code>g /= 2</code>	<code>g = g / 2</code>	4.5 to g
<code>//=</code>	<code>g //= 2</code>	<code>g = g // 2</code>	4 to g
<code>%=</code>	<code>h %= 9</code>	<code>h = h % 9</code>	3 to h

✓ Self Check

1 (*Fill-In*) If x is 7, the value of x after evaluating $x * = 5$ is _____.

Answer: 35.

2 (*IPython Session*) Create a variable x with the value 12. Use an exponentiation augmented assignment statement to square x 's value. Show x 's new value.

Answer:

```
In [1]: x = 12
```

```
In [2]: x **= 2
```

```
In [3]: x
Out[3]: 144
```

3.10 Program Development: Sequence-Controlled Repetition

Experience has shown that the most challenging part of solving a problem on a computer is developing an algorithm for the solution. As you'll see, once a correct algorithm has been specified, creating a working Python program from the algorithm is typically straightforward. This section and the next present problem solving and program development by creating scripts that solve two class-averaging problems.

3.10.1 Requirements Statement

A **requirements statement** describes *what* a program is supposed to do, but not *how* the program should do it. Consider the following simple requirements statement:

A class of ten students took a quiz. Their grades (integers in the range 0 – 100) are 98, 76, 71, 87, 83, 90, 57, 79, 82, 94. Determine the class average on the quiz.

Once you know the problem's requirements, you can begin creating an algorithm to solve it. Then, you can implement that solution as a program.

The algorithm for solving this problem must:

- 1.** Keep a running total of the grades.
- 2.** Calculate the average—the total of the grades divided by the number of grades.
- 3.** Display the result.

For this example, we'll place the 10 grades in a list. You also could input the grades from a user at the keyboard (as we'll do in the next example) or read them from a file (as you'll see how to do in the "Files and Exceptions" chapter). We also show you how to read data from SQL and NoSQL databases in later chapters.

3.10.2 Pseudocode for the Algorithm

The following pseudocode lists the actions to execute and specifies the order in which they should execute:

Set total to zero
Set grade counter to zero
Set grades to a list of the ten grades

For each grade in the grades list:
 Add the grade to the total
 Add one to the grade counter

Set the class average to the total divided by the number of grades
Display the class average

Note the mentions of *total* and *grade counter*. In Fig. 3.1's script, the variable `total` (line 5) stores the grade values' running total, and `grade_counter` (line 6) counts the number of grades we've processed. We'll use these to calculate the average. Variables for totaling and counting normally are initialized to zero before they're used, as we do in lines 5 and 6.

3.10.3 Coding the Algorithm in Python

The following script implements the pseudocode algorithm.

```

1  # fig03_01.py
2  """Class average program with sequence-controlled repetition."""
3
4  # initialization phase
5  total = 0 # sum of grades
6  grade_counter = 0
7  grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94] # list of 10 grades
8
9  # processing phase
10 for grade in grades:
11     total += grade # add current grade to the running total
12     grade_counter += 1 # indicate that one more grade was processed
13
14 # termination phase
15 average = total / grade_counter
16 print(f'Class average is {average}')
```

```
Class average is 81.7
```

Fig. 3.1 | Class average program with sequence-controlled repetition.

Execution Phases

We used blank lines and comments to break the script into three **execution phases**—initialization, processing and termination:

- The **initialization phase** creates the variables needed to process the grades and set these variables to appropriate initial values.
- The **processing phase** processes the grades, calculating the running total and counting the number of grades processed so far.
- The **termination phase** calculates and displays the class average.

Many scripts can be **decomposed** (that is, broken apart) into these three phases.

Initialization Phase

Lines 5–6 create the variables `total` and `grade_counter` and initialize each to 0. Line 7

```
grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94] # list of 10 grades
```

creates the variable `grades` and initializes it with a list of 10 integer grades.

Processing Phase

The `for` statement processes each grade in the list `grades`. Line 11 adds the current grade to the `total`. Then, line 12 adds 1 to the variable `grade_counter` to keep track of the number of grades processed so far. Repetition terminates when all 10 grades in the list have been processed. This is called **definite repetition** because the number of repetitions is known before the loop begins executing. In this case, it's the number of elements in the list `grades`. The *Style Guide for Python Code* recommends placing a blank line above and below each control statement (as in lines 8 and 13).

Termination Phase

When the `for` statement terminates, line 15 calculates the average and assigns it to the variable `average`. Then line 16 displays `average`. Later in this chapter, we use functional-style programming features to calculate the average of a list's items more concisely.

3.10.4 Introduction to Formatted Strings

Line 16 uses the following simple **f-string** (short for **formatted string**) to format this script's result by inserting the value of `average` into a string:

```
f'Class average is {average}'
```

The letter `f` before the string's opening quote indicates it's an f-string. You specify where to insert values by using placeholders delimited by curly braces (`{` and `}`). The placeholder

```
{average}
```

converts the variable `average`'s value to a string representation, then replaces `{average}` with that **replacement text**. Replacement-text expressions may contain values, variables or other expressions, such as calculations or function calls. In line 16, we could have used `total / grade_counter` in place of `average`, eliminating the need for line 15.



Self Check

1 (Fill-In) A(n) _____ describes *what* a program is supposed to do, but not *how* the program should do it.

Answer: requirements statement.

2 (Fill-In) Many of the scripts you'll write can be decomposed into three phases: _____, _____ and _____.

Answer: initialization, processing, termination.

3 (IPython Session) Display an f-string in which you insert the values of the variables `number1` (7) and `number2` (5) and their product. The displayed string should be

```
7 times 5 is 35
```

Answer:

```
In [1]: number1 = 7
```

```
In [2]: number2 = 5
```

```
In [3]: print(f'{number1} times {number2} is {number1 * number2}')  
7 times 5 is 35
```

3.11 Program Development: Sentinel-Controlled Repetition

Let's generalize the class-average problem. Consider the following requirements statement:

Develop a class-averaging program that processes an arbitrary number of grades each time the program executes.

In the first class-average example, we knew in advance the 10 grades to process. The requirements statement does not state what the grades are or how many there are, so we're going to have the user enter the grades into the program. The program processes an arbitrary number of grades. How can the program determine when to stop processing grades so that it can move on to calculate and display the class average?

One way to solve this problem is to use a special value called a **sentinel value** (also called a **signal value**, a **dummy value** or a **flag value**) to indicate “end of data entry.” This is a bit like the way a caboose “marks” the end of a train. The user enters grades one at a time until all the grades have been entered. The user then enters the sentinel value to indicate that there are no more grades. **Sentinel-controlled repetition** is often called **indefinite repetition** because the number of repetitions is *not* known before the loop begins executing.

A sentinel value must not be confused with any acceptable input value. Grades on a quiz are typically nonnegative integers between 0 and 100, so the value -1 is an acceptable sentinel value for this problem. Thus, a run of the class-average program might process a stream of inputs such as 95, 96, 75, 74, 89 and -1 . The program would then compute and print the class average for the grades 95, 96, 75, 74 and 89. The sentinel value -1 should *not* enter into the averaging calculation.

Developing the Pseudocode Algorithm with Top-Down, Stepwise Refinement

We approach this class-average problem with a technique called **top-down, stepwise refinement**. We begin with a pseudocode representation of the top:

Determine the class average for the quiz

The top is a single statement that conveys the program's overall function. Although it's a *complete* representation of a program, the top rarely conveys enough detail from which to write a program. The *top* specifies *what* should be done, but not *how* to implement it. So we begin the **refinement process**. We decompose the top into a sequence of smaller tasks—a process sometimes called **divide and conquer**. This results in the following **first refinement**:

Initialize variables

Input, sum and count the quiz grades

Calculate and display the class average

Each refinement represents the *complete* algorithm—only the level of detail varies. In this refinement, the three pseudocode statements happen to correspond to the three execution phases described in the preceding section. The algorithm does not yet provide enough detail for us to write the Python program. So, we continue with the next refinement.

Second Refinement

To proceed to the **second refinement**, we commit to specific variables. The program needs to maintain

- a grade variable in which each successive user input will be stored,
- a running total of the grades,
- a count of how many grades have been processed and
- a variable that contains the calculated average.

The pseudocode statement

Initialize variables

can be refined as follows:

Initialize total to zero

Initialize grade counter to zero

Only the variables *total* and *grade counter* need to be initialized before they're used. We do not initialize the variables for the user input and calculated average. Their values will be replaced each time we input a grade from the user and when we calculate the class average, respectively. We'll create these variables when they're needed.

The next pseudocode statement requires a loop that successively inputs each grade:

Input, sum and count the quiz grades

We do not know how many grades will be entered, so we use sentinel-controlled repetition. The user enters legitimate grades successively. After the last legitimate grade has been entered, the user enters the sentinel value. The program tests for the sentinel value after each grade is input and terminates the loop when the sentinel has been entered. The second refinement of the preceding pseudocode statement is

Input the first grade (possibly the sentinel)

While the user has not entered the sentinel

Add this grade into the running total

Add one to the grade counter

Input the next grade (possibly the sentinel)

The pseudocode statement

Calculate and display the class average

can be refined as follows:

If the counter is not equal to zero

Set the average to the total divided by the grade counter

Display the average

Else

Display "No grades were entered"

Notice that we're testing for the possibility of division by zero. If undetected, this would cause a fatal logic error. In the "Files and Exceptions" chapter, we discuss how to write programs that recognize such exceptions and take appropriate actions.

The following is the class-average problem's complete second refinement:

```
Initialize total to zero
Initialize grade counter to zero

Input the first grade (possibly the sentinel)
While the user has not entered the sentinel
    Add this grade into the running total
    Add one to the grade counter
    Input the next grade (possibly the sentinel)

If the counter is not equal to zero
    Set the average to the total divided by the counter
    Display the average
Else
    Display "No grades were entered"
```

Sometimes more than two refinements are necessary. You stop refining when there is enough detail for you to convert the pseudocode to Python. We include blank lines for readability. Here, they happen to separate the algorithm into the three popular execution phases.

Implementing Sentinel-Controlled Iteration

The following script implements the pseudocode algorithm and shows a sample execution in which the user enters three grades and the sentinel value.

```
1  # fig03_02.py
2  """Class average program with sentinel-controlled iteration."""
3
4  # initialization phase
5  total = 0 # sum of grades
6  grade_counter = 0 # number of grades entered
7
8  # processing phase
9  grade = int(input('Enter grade, -1 to end: ')) # get one grade
10
11 while grade != -1:
12     total += grade
13     grade_counter += 1
14     grade = int(input('Enter grade, -1 to end: '))
15
16 # termination phase
17 if grade_counter != 0:
18     average = total / grade_counter
19     print(f'Class average is {average:.2f}')
20 else:
21     print('No grades were entered')
```

Fig. 3.2 | Class average program with sentinel-controlled iteration. (Part 1 of 2.)

```

Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 72
Enter grade, -1 to end: -1
Class average is 85.67

```

Fig. 3.2 | Class average program with sentinel-controlled iteration. (Part 2 of 2.)

Program Logic for Sentinel-Controlled Repetition

In sentinel-controlled repetition, the program reads the first value (line 9) before reaching the `while` statement. Line 9 demonstrates why we did not create the variable `grade` until we needed it in the program. If we had initialized it, that value would have been replaced immediately by this assignment.

The value input in line 9 determines whether the program's flow of control should enter the `while`'s suite (lines 12–14). If the condition in line 11 is `False`, the user entered the sentinel value (`-1`), so the suite does not execute because the user did not enter any grades. If the condition is `True`, the suite executes, adding the `grade` value to the `total` and incrementing the `grade_counter`. Next, line 14 inputs another grade from the user. Then, the `while`'s condition (line 11) is tested again, using the most recent `grade` entered by the user. The value of `grade` is always input immediately before the program tests the `while` condition, so we can determine whether the value just input is the sentinel before processing that value as a grade. When the sentinel value is input, the loop terminates, and the program does not add `-1` to the total. In a sentinel-controlled loop that performs user input, any prompts (lines 9 and 14) should remind the user of the sentinel value.

After the loop terminates, the `if...else` statement (lines 17–21) executes. Line 17 determines whether the user entered any grades. If not, the `else` part (lines 20–21) executes and displays the message 'No grades were entered' and the program terminates.

Formatting the Class Average with Two Decimal Places

This example formatted the class average with two digits to the right of the decimal point. In an f-string, you can optionally follow a replacement-text expression with a colon (`:`) and a **format specifier** that describes how to format the replacement text. The format specifier `.2f` (line 19) formats the average as a floating-point number (`f`) with two digits to the right of the decimal point (`.2`). In this example, the sum of the grades was 257, which, when divided by 3, yields 85.666666666.... Formatting the average with `.2f` *rounds* it to the hundredths position, producing the replacement text 85.67. An average with only one digit to the right of the decimal point would be formatted with a **trailing zero** (e.g., 85.50). The chapter “Strings: A Deeper Look” discusses many string-formatting features.

Control-Statement Stacking

In this example, notice that control statements are stacked in sequence. The `while` statement (lines 11–14) is followed immediately by an `if...else` statement (lines 17–21).



Self Check

I (*Fill-In*) Sentinel-controlled repetition is called _____ because the number of repetitions is not known before the loop begins executing.

Answer: indefinite repetition.

2 (*True/False*) Sentinel-controlled repetition uses a counter variable to control the number of times a set of instructions executes.

Answer: False. Sentinel-control repetition terminates repetition when the sentinel value is encountered.

3.12 Program Development: Nested Control Statements

Let's work through another complete problem. Once again, we plan the algorithm using pseudocode and top-down, stepwise refinement and we develop a corresponding Python script. Consider the following requirements statement:

A college offers a course that prepares students for the state licensing exam for real-estate brokers. Last year, several of the students who completed this course took the licensing examination. The college wants to know how well its students did on the exam. You have been asked to write a program to summarize the results. You have been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam and a 2 if the student failed.

Your program should analyze the results of the exam as follows:

- 1. Input each test result (i.e., a 1 or a 2). Display the message "Enter result" each time the program requests another test result.*
- 2. Count the number of test results of each type.*
- 3. Display a summary of the test results indicating the number of students who passed and the number of students who failed.*
- 4. If more than eight students passed the exam, display "Bonus to instructor."*

After reading the requirements statement carefully, we make the following observations about the problem:

- 1.** The program must process 10 test results. We'll use a `for` statement and the `range` function to control repetition.
- 2.** Each test result is a number—either a 1 or a 2. Each time the program reads a test result, the program must determine if the number is a 1 or a 2. We test for a 1 in our algorithm. If the number is not a 1, we assume that it's a 2. (An exercise at the end of the chapter considers the consequences of this assumption.)
- 3.** We'll use two counters—one to count the number of students who passed the exam and one to count the number of students who failed.
- 4.** After the script processes all the results, it must decide if more than eight students passed the exam so that it can bonus the instructor.

Top-Down, Stepwise Refinement

We begin with a pseudocode representation of the top:

Analyze exam results and decide whether instructor should receive a bonus

Once again, the top is a *complete* representation of the program, but several refinements are likely to be needed before the pseudocode can evolve naturally into a Python program.

First Refinement

Our first refinement is

```
Initialize variables
Input the ten exam grades and count passes and failures
Summarize the exam results and decide whether instructor should receive a bonus
```

Here, too, even though we have a *complete* representation of the entire program, further refinement is necessary. Note again that this first refinement happens to correspond to the three-execution-phases model.

Second Refinement

We now commit to specific variables. We need counters to record the passes and failures, and a variable to store the user input. The pseudocode statement

```
Initialize variables
```

can be refined as follows:

```
Initialize passes to zero
Initialize failures to zero
```

Only the counters for the number of passes and number of failures need to be initialized. The pseudocode statement

```
Input the ten exam grades and count passes and failures
```

requires a loop that successively inputs the result of each exam. Here it's known in advance that there are ten exam results, so the `for` statement and the `range` function are appropriate. Inside the loop (that is, *nested* within the loop), an `if...else` statement determines whether each exam result is a pass or a failure and increments the appropriate counter. The refinement of the preceding pseudocode statement is

```
For each of the ten students
    Input the next exam result

    If the student passed
        Add one to passes
    Else
        Add one to failures
```

The blank line before the *If...Else* improves readability.

The pseudocode statement

```
Summarize the exam results and decide whether instructor should receive a bonus
```

may be refined as follows:

```
Display the number of passes
Display the number of failures

If more than eight students passed
    Display "Bonus to instructor"
```

Complete Pseudocode Algorithm

The pseudocode is now sufficiently refined for conversion to Python—the complete second refinement is shown below:

```

Initialize passes to zero
Initialize failures to zero

For each of the ten students
    Input the next exam result

    If the student passed
        Add one to passes
    Else
        Add one to failures

Display the number of passes
Display the number of failures

If more than eight students passed
    Display "Bonus to instructor"
  
```

Implementing the Algorithm

The following script implements the algorithm and is followed by two sample executions. Once again, notice that the Python code closely resembles the pseudocode. Lines 9–16 loop 10 times, inputting and processing one exam result each time. The `if...else` statement (lines 13–16) that processes each result is *nested* in the `for` statement—that is, it's part of the `for` statement's suite. If the result is 1, we add 1 to passes; otherwise, we assume the result is 2 and add 1 to failures. After inputting 10 values, the loop terminates and lines 19 and 20 display passes and failures. Lines 22–23 determine whether more than eight students passed the exam and, if so, display 'Bonus to instructor'.

```

1  # fig03_03.py
2  """Using nested control statements to analyze examination results."""
3
4  # initialize variables
5  passes = 0 # number of passes
6  failures = 0 # number of failures
7
8  # process 10 students
9  for student in range(10):
10     # get one exam result
11     result = int(input('Enter result (1=pass, 2=fail): '))
12
13     if result == 1:
14         passes = passes + 1
15     else:
16         failures = failures + 1
17
  
```

Fig. 3.3 | Analysis of examination results. (Part I of 2.)

```

18 # termination phase
19 print('Passed:', passes)
20 print('Failed:', failures)
21
22 if passes > 8:
23     print('Bonus to instructor')

```

```

Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Passed: 6
Failed: 4

```

```

Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 2
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Enter result (1=pass, 2=fail): 1
Passed: 9
Failed: 1
Bonus to instructor

```

Fig. 3.3 | Analysis of examination results. (Part 2 of 2.)



Self Check

I (*IPython Session*) Use a for statement to input two integers. Use a nested if...else statement to display whether each value is even or odd. Enter 10 and 7 to test your code.

Answer:

```

In [1]: for count in range(2):
...:     value = int(input('Enter an integer: '))
...:     if value % 2 == 0:
...:         print(f'{value} is even')
...:     else:
...:         print(f'{value} is odd')
...:
Enter an integer: 10
10 is even
Enter an integer: 7
7 is odd

```

3.13 Built-In Function range: A Deeper Look

Function `range` also has two- and three-argument versions. As you've seen, `range`'s one-argument version produces a sequence of consecutive integers from 0 up to, but not including, the argument's value. Function `range`'s two-argument version produces a sequence of consecutive integers from its first argument's value up to, but not including, the second argument's value, as in:

```
In [1]: for number in range(5, 10):
...:     print(number, end=' ')
...:
5 6 7 8 9
```

Function `range`'s three-argument version produces a sequence of integers from its first argument's value up to, but not including, the second argument's value, *incrementing* by the third argument's value, which is known as the **step**:

```
In [2]: for number in range(0, 10, 2):
...:     print(number, end=' ')
...:
0 2 4 6 8
```

If the third argument is negative, the sequence progresses from the first argument's value *down* to, but not including the second argument's value, *decrementing* by the third argument's value, as in:

```
In [3]: for number in range(10, 0, -2):
...:     print(number, end=' ')
...:
10 8 6 4 2
```

✓ Self Check

1 (*True/False*) Function call `range(1, 10)` generates the sequence 1 through 10.

Answer: False. Function call `range(1, 10)` generates the sequence 1 through 9.

2 (*IPython Session*) What happens if you try to print the items in `range(10, 0, 2)`?

Answer: Nothing displays because the step is not negative (this is not a fatal error):

```
In [1]: for number in range(10, 0, 2):
...:     print(number, end=' ')
...:
```

```
In [2]:
```

3 (*IPython Session*) Use a for statement, `range` and `print` to display on one line the sequence of values 99 88 77 66 55 44 33 22 11 0, each separated by one space.

Answer:

```
In [3]: for number in range(99, -1, -11):
...:     print(number, end=' ')
...:
99 88 77 66 55 44 33 22 11 0
```

4 (*IPython Session*) Use `for` and `range` to sum the even integers from 2 through 100, then display the sum.

Answer:

```
In [4]: total = 0

In [5]: for number in range(2, 101, 2):
...:     total += number
...:

In [6]: total
Out[6]: 2550
```

3.14 Using Type Decimal for Monetary Amounts

In this section, we introduce `Decimal` capabilities for precise monetary calculations. If you enter banking or other fields that require the accuracy provided by type `Decimal`, you should investigate `Decimal`'s capabilities in depth.

For most scientific and other mathematical applications that use numbers with decimal points, Python's built-in floating-point numbers work well. For example, when we speak of a "normal" body temperature of 98.6, we do not need to be precise to a large number of digits. When we view the temperature on a thermometer and read it as 98.6, the actual value may be 98.5999473210643. The point here is that calling this number 98.6 is adequate for most body-temperature applications.

Floating-point values are stored in binary format (we introduced binary in the first chapter and discuss it in depth in the online "Number Systems" appendix). Some floating-point values are represented only approximately when they're converted to binary. For example, consider the variable `amount` with the dollars-and-cents value 112.31. If you display `amount`, it appears to have the exact value you assigned to it:

```
In [1]: amount = 112.31

In [2]: print(amount)
112.31
```

However, if you print `amount` with 20 digits of precision to the right of the decimal point, you can see that the actual floating-point value in memory is not exactly 112.31—it's only an approximation:

```
In [3]: print(f'{amount:.20f}')
112.31000000000000227374
```

Many applications require *precise* representation of numbers with decimal points. Institutions like banks that deal with millions or even billions of transactions per day have to tie out their transactions "to the penny." Floating-point numbers can represent some but not all monetary amounts with to-the-penny precision.

The **Python Standard Library**² provides many predefined capabilities you can use in your Python code to avoid "reinventing the wheel." For monetary calculations and other applications that require precise representation and manipulation of numbers with decimal points, the Python Standard Library provides type **Decimal**, which uses a special coding scheme to solve the problem of to-the-penny precision. That scheme requires additional memory to hold the numbers and additional processing time to perform calcu-

2. <https://docs.python.org/3.7/library/index.html>.

lations but provides the to-the-penny precision required for monetary calculations. Banks also have to deal with other issues such as using a fair rounding algorithm when they're calculating daily interest on accounts. Type Decimal offers such capabilities.³

Importing Type Decimal from the decimal Module

We've used several built-in types—**int** (for integers, like 10), **float** (for floating-point numbers, like 7.5) and **str** (for strings like 'Python'). The Decimal type is not built into Python. Rather, it's part of the Python Standard Library, which is divided into **modules**—groups of related capabilities. The **decimal** module defines type Decimal and its capabilities.

To use capabilities from a module, you must first **import** the entire module, as in

```
import decimal
```

and refer to the Decimal type as decimal.Decimal, or you must indicate a specific capability to import using **from...import**, as we do here:

```
In [4]: from decimal import Decimal
```

This imports only the type Decimal from the decimal module so that you can use it in your code. We'll discuss other import forms beginning in the next chapter.

Creating Decimals

You typically create a Decimal from a string:

```
In [5]: principal = Decimal('1000.00')
```

```
In [6]: principal
Out[6]: Decimal('1000.00')
```

```
In [7]: rate = Decimal('0.05')
```

```
In [8]: rate
Out[8]: Decimal('0.05')
```

We'll soon use the variables principal and rate in a compound-interest calculation.

Decimal Arithmetic

Decimals support the standard arithmetic operators +, -, *, /, //, ** and %, as well as the corresponding augmented assignments:

```
In [9]: x = Decimal('10.5')
```

```
In [10]: y = Decimal('2')
```

```
In [11]: x + y
Out[11]: Decimal('12.5')
```

```
In [12]: x // y
Out[12]: Decimal('5')
```

```
In [13]: x += y
```

```
In [14]: x
Out[14]: Decimal('12.5')
```

3. For more decimal module features, visit <https://docs.python.org/3.7/library/decimal.html>.

You may perform arithmetic between Decimals and integers, but not between Decimals and floating-point numbers.

Compound-Interest Problem Requirements Statement

Let's compute compound interest using the Decimal type for *precise* monetary calculations. Consider the following requirements statement:

A person invests \$1000 in a savings account yielding 5% interest. Assuming that the person leaves all interest on deposit in the account, calculate and display the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal),

r is the annual interest rate,

n is the number of years and

a is the amount on deposit at the end of the *n*th year.

Calculating Compound Interest

To solve this problem, let's use variables `principal` and `rate` that we defined in snippets [5] and [7], and a `for` statement that performs the interest calculation for each of the 10 years the money remains on deposit. For each year, the loop displays a formatted string containing the year number and the amount on deposit at the end of that year:

```
In [15]: for year in range(1, 11):
...:     amount = principal * (1 + rate) ** year
...:     print(f'{year:>2}{amount:>10.2f}')
...:
1    1050.00
2    1102.50
3    1157.62
4    1215.51
5    1276.28
6    1340.10
7    1407.10
8    1477.46
9    1551.33
10   1628.89
```

The algebraic expression $(1 + r)^n$ from the requirements statement is written as

```
(1 + rate) ** year
```

where variable `rate` represents *r* and variable `year` represents *n*.

Formatting the Year and Amount on Deposit

The statement

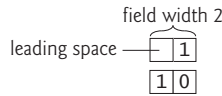
```
print(f'{year:>2}{amount:>10.2f}')
```

uses an f-string with two placeholders to format the loop's output.

The placeholder

```
{year:>2}
```

uses the format specifier `>2` to indicate that year's value should be **right aligned** (`>`) in a field of width 2—the **field width** specifies the number of character positions to use when displaying the value. For the single-digit year values 1–9, the format specifier `>2` displays a space character followed by the value, thus right aligning the years in the first column. The following diagram shows the numbers 1 and 10 each formatted in a field width of 2:

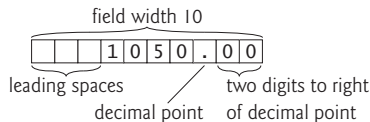


You can **left align** values with `<`.

The format specifier `10.2f` in the placeholder

```
{amount:>10.2f}
```

formats `amount` as a floating-point number (`f`) right aligned (`>`) in a field width of 10 with a decimal point and two digits to the right of the decimal point (`.2`). Formatting the amounts this way *aligns their decimal points vertically*, as is typical with monetary amounts. In the 10 character positions, the three rightmost characters are the number's decimal point followed by the two digits to its right. The remaining seven character positions are the leading spaces and the digits to the decimal point's left. In this example, all the dollar amounts have four digits to the left of the decimal point, so each number is formatted with three leading spaces. The following diagram shows the formatting for the value 1050.00:



✓ Self Check

1 (Fill-In) A field width specifies the _____ to use when displaying a value.

Answer: number of character positions.

2 (IPython Session) Assume that the tax on a restaurant bill is 6.25% and that the bill amount is \$37.45. Use type `Decimal` to calculate the bill total, then print the result with two digits to the right of the decimal point.

Answer:

```
In [1]: from decimal import Decimal
```

```
In [2]: print(f"{Decimal('37.45') * Decimal('1.0625'):.2f}")
39.79
```

3.15 break and continue Statements

The `break` and `continue` statements alter a loop's flow of control. Executing a **break** statement in a `while` or `for` immediately exits that statement. In the following code, `range` produces the integer sequence 0–99, but the loop terminates when number is 10:

```
In [1]: for number in range(100):
...:     if number == 10:
...:         break
...:     print(number, end=' ')
...:
0 1 2 3 4 5 6 7 8 9
```

In a script, execution would continue with the next statement after the `for` loop. The `while` and `for` statements each have an optional `else` clause that executes only if the loop terminates normally—that is, not as a result of a `break`. We explore this in the exercises.

Executing a **`continue`** statement in a `while` or `for` loop skips the remainder of the loop's suite. In a `while`, the condition is then tested to determine whether the loop should continue executing. In a `for`, the loop processes the next item in the sequence (if any):

```
In [2]: for number in range(10):
...:     if number == 5:
...:         continue
...:     print(number, end=' ')
...:
0 1 2 3 4 6 7 8 9
```

3.16 Boolean Operators `and`, `or` and `not`

The conditional operators `>`, `<`, `>=`, `<=`, `==` and `!=` can be used to form simple conditions such as `grade >= 60`. To form more complex conditions that combine simple conditions, use the `and`, `or` and `not` Boolean operators.

Boolean Operator `and`

To ensure that two conditions are *both* `True` before executing a control statement's suite, use the **Boolean `and` operator** to combine the conditions. The following code defines two variables, then tests a condition that's `True` if and only if *both* simple conditions are `True`—if either (or both) of the simple conditions is `False`, the entire `and` expression is `False`:

```
In [1]: gender = 'Female'

In [2]: age = 70

In [3]: if gender == 'Female' and age >= 65:
...:     print('Senior female')
...:
Senior female
```

The `if` statement has two simple conditions:

- `gender == 'Female'` determines whether a person is a female and
- `age >= 65` determines whether that person is a senior citizen.

The simple condition to the left of the `and` operator evaluates first because `==` has higher precedence than `and`. If necessary, the simple condition to the right of `and` evaluates next, because `>=` has higher precedence than `and`. (We'll discuss shortly why the right side of an `and` operator evaluates *only* if the left side is `True`.) The entire `if` statement condition is `True` if and only if *both* of the simple conditions are `True`. The combined condition can be made clearer by adding redundant (unnecessary) parentheses

```
(gender == 'Female') and (age >= 65)
```

The table below summarizes the and operator by showing all four possible combinations of False and True values for *expression1* and *expression2*—such tables are called *truth tables*:

<i>expression1</i>	<i>expression2</i>	<i>expression1 and expression2</i>
False	False	False
False	True	False
True	False	False
True	True	True

Boolean Operator or

Use the **Boolean or operator** to test whether one *or* both of two conditions are True. The following code tests a condition that's True if either *or* both simple conditions are True—the entire condition is False only if *both* simple conditions are False:

```
In [4]: semester_average = 83

In [5]: final_exam = 95

In [6]: if semester_average >= 90 or final_exam >= 90:
...:     print('Student gets an A')
...:
Student gets an A
```

Snippet [6] also contains two simple conditions:

- `semester_average >= 90` determines whether a student's average was an A (90 or above) during the semester, and
- `final_exam >= 90` determines whether a student's final-exam grade was an A.

The truth table below summarizes the Boolean or operator. Operator and has higher precedence than or.

<i>expression1</i>	<i>expression2</i>	<i>expression1 or expression2</i>
False	False	False
False	True	True
True	False	True
True	True	True

Improving Performance with Short-Circuit Evaluation

Python stops evaluating an and expression as soon as it knows whether the entire condition is False. Similarly, Python stops evaluating an or expression as soon as it knows whether the entire condition is True. This is called *short-circuit evaluation*. So the condition

```
gender == 'Female' and age >= 65
```

stops evaluating immediately if `gender` is not equal to 'Female' because the entire expression must be False. If `gender` is equal to 'Female', execution continues, because the entire expression will be True if the age is greater than or equal to 65.

Similarly, the condition

```
semester_average >= 90 or final_exam >= 90
```

stops evaluating immediately if `semester_average` is greater than or equal to 90 because the entire expression must be True. If `semester_average` is less than 90, execution continues, because the expression could still be True if the `final_exam` is greater than or equal to 90.

In operator expressions that use `and`, make the condition that’s more likely to be False the leftmost condition. In `or` operator expressions, make the condition that’s more likely to be True the leftmost condition. These can reduce a program’s execution time.

Boolean Operator not

The **Boolean not operator** “reverses” the meaning of a condition—True becomes False and False becomes True. This is a **unary operator**—it has only *one* operand. You place the `not` operator before a condition to choose a path of execution if the original condition (without the `not` operator) is False, such as in the following code:

```
In [7]: grade = 87

In [8]: if not grade == -1:
...:     print('The next grade is', grade)
...:
The next grade is 87
```

Often, you can avoid using `not` by expressing the condition in a more “natural” or convenient manner. For example, the preceding `if` statement can also be written as follows:

```
In [9]: if grade != -1:
...:     print('The next grade is', grade)
...:
The next grade is 87
```

The truth table below summarizes the `not` operator.

expression	not expression
False	True
True	False

The following table shows the precedence and grouping of the operators introduced so far, from top to bottom, in decreasing order of precedence.

Operators	Grouping
()	left to right
**	right to left
* / // %	left to right
+ -	left to right
< <= > >= == !=	left to right
not	left to right
and	left to right
or	left to right

 Self Check

I (*IPython Session*) Assume that $i = 1$, $j = 2$, $k = 3$ and $m = 2$. What does each of the following conditions display?

- a) $(i \geq 1)$ and $(j < 4)$
- b) $(m \leq 99)$ and $(k < m)$
- c) $(j \geq i)$ or $(k == m)$
- d) $(k + m < j)$ or $(3 - j \geq k)$
- e) not $(k > m)$

Answer:

```
In [1]: i = 1
In [2]: j = 2
In [3]: k = 3
In [4]: m = 2

In [5]: (i >= 1) and (j < 4)
Out[5]: True

In [6]: (m <= 99) and (k < m)
Out[6]: False

In [7]: (j >= i) or (k == m)
Out[7]: True

In [8]: (k + m < j) or (3 - j >= k)
Out[8]: False

In [9]: not (k > m)
Out[9]: False
```

3.17 Intro to Data Science: Measures of Central Tendency—Mean, Median and Mode

Here we continue our discussion of using statistics to analyze data with several additional descriptive statistics, including:

- **mean**—the average value in a set of values.
- **median**—the middle value when all the values are arranged in sorted order.
- **mode**—the most frequently occurring value.

These are **measures of central tendency**—each is a way of producing a single value that represents a “central” value in a set of values, i.e., a value which is in some sense typical of the others.

Let’s calculate the mean, median and mode on a list of integers. The following session creates a list called `grades`, then uses the built-in **sum** and **len** functions to calculate the mean “by hand”—`sum` calculates the total of the grades (397) and `len` returns the number of grades (5):

```
In [1]: grades = [85, 93, 45, 89, 85]
```

```
In [2]: sum(grades) / len(grades)
Out[2]: 79.4
```

The previous chapter mentioned the descriptive statistics count and sum—implemented in Python as the built-in functions `len` and `sum`. Like functions `min` and `max` (introduced in the preceding chapter), `sum` and `len` are both examples of functional-style programming *reductions*—they reduce a collection of values to a single value—the sum of those values and the number of values, respectively. In Fig. 3.1’s class-average example, we could have deleted lines 10–15 and replaced `average` in line 16 with snippet [2]’s calculation.

The Python Standard Library’s **statistics module** provides functions for calculating the mean, median and mode—these, too, are reductions. To use these capabilities, first import the `statistics` module:

```
In [3]: import statistics
```

Then, you can access the module’s functions with “`statistics.`” followed by the name of the function to call. The following calculates the `grades` list’s mean, median and mode, using the `statistics` module’s **mean**, **median** and **mode** functions:

```
In [4]: statistics.mean(grades)
Out[4]: 79.4
```

```
In [5]: statistics.median(grades)
Out[5]: 85
```

```
In [6]: statistics.mode(grades)
Out[6]: 85
```

Each function’s argument must be an *iterable*—in this case, the list `grades`. To confirm that the median and mode are correct, you can use the built-in **sorted function** to get a copy of `grades` with its values arranged in increasing order:

```
In [7]: sorted(grades)
Out[7]: [45, 85, 85, 89, 93]
```

The `grades` list has an odd number of values (5), so `median` returns the middle value (85). If the list’s number of values is even, `median` returns the *average* of the *two* middle values. Studying the sorted values, you can see that 85 is the mode because it occurs most frequently (twice). The `mode` function causes a `StatisticsError` for lists like

```
[85, 93, 45, 89, 85, 93]
```

in which there are two or more “most frequent” values. Such a set of values is said to be **bimodal**. Here, both 85 and 93 occur twice. We’ll say more about mean, median and mode in the Intro to Data Science exercises at the end of the chapter.



Self Check

1 (Fill-In) The _____ statistic indicates the average value in a set of values.

Answer: mean.

2 (Fill-In) The _____ statistic indicates the most frequently occurring value in a set of values.

Answer: mode.

3 (Fill-In) The _____ statistic indicates the middle value in a set of values.

Answer: median.

4 (IPython Session) For the values 47, 95, 88, 73, 88 and 84, use the `statistics` module to calculate the mean, median and mode.

Answer:

```
In [1]: import statistics
In [2]: values = [47, 95, 88, 73, 88, 84]
In [3]: statistics.mean(values)
Out[3]: 79.16666666666667

In [4]: statistics.median(values)
Out[4]: 86.0

In [5]: statistics.mode(values)
Out[5]: 88
```

3.18 Wrap-Up

In this chapter, we discussed Python's control statements, including `if`, `if...else`, `if...elif...else`, `while`, `for`, `break` and `continue`. We used pseudocode and top-down, stepwise refinement to develop several algorithms. You saw that many simple algorithms often have three execution phases—initialization, processing and termination.

You saw that the `for` statement performs sequence-controlled iteration—it processes each item in an iterable, such as a range of integers, a string or a list. You used the built-in function `range` to generate sequences of integers from 0 up to, but not including, its argument, and to determine how many times a `for` statement iterates. You used sentinel-controlled repetition with the `while` statement to create a loop that continues executing until a sentinel value is encountered. You used built-in function `range`'s two-argument version to generate sequences of integers from the first argument's value up to, but not including, the second argument's value. You also used the three-argument version in which the third argument indicated the step between integers in a range.

We introduced the `Decimal` type for precise monetary calculations and used it to calculate compound interest. You used f-strings and various format specifiers to create formatted output. We introduced the `break` and `continue` statements for altering the flow of control in loops. We discussed the Boolean operators `and`, `or` and `not` for creating conditions that combine simple conditions.

Finally, we continued our discussion of descriptive statistics by introducing measures of central tendency—mean, median and mode—and calculating them with functions from the Python Standard Library's `statistics` module.

In the next chapter, you'll create custom functions and use existing functions from Python's `math` and `random` modules. We show several predefined functional-programming reductions. You'll learn more of Python's functional-programming capabilities.

Exercises

Unless specified otherwise, use IPython sessions for each exercise.

3.1 (Validating User Input) Modify the script of Fig. 3.3 to validate its inputs. For any input, if the value entered is other than 1 or 2, keep looping until the user enters a correct

value. Use one counter to keep track of the number of passes, then calculate the number of failures after all the user's inputs have been received.

3.2 (*What's Wrong with This Code?*) What is wrong with the following code?

```
a = b = 7
print('a =', a, '\nb =', b)
```

First, answer the question, then check your work in an IPython session.

3.3 (*What Does This Code Do?*) What does the following program print?

```
for row in range(10):
    for column in range(10):
        print('<' if row % 2 == 1 else '>', end='')
    print()
```

3.4 (*Fill in the Missing Code*) In the code below

```
for ***:
    for ***:
        print('@')
    print()
```

replace the *** so that when you execute the code, it displays two rows, each containing seven @ symbols, as in:

```
@@@@@@@
@@@@@@@
```

3.5 (*if...else Statements*) Reimplement the script of Fig. 2.1 using three if...else statements rather than six if statements. [Hint: For example, think of == and != as “opposite” tests.]

3.6 (*Turing Test*) The great British mathematician Alan Turing proposed a simple test to determine whether machines could exhibit intelligent behavior. A user sits at a computer and does the same text chat with a human sitting at a computer and a computer operating by itself. The user doesn't know if the responses are coming back from the human or the independent computer. If the user can't distinguish which responses are coming from the human and which are coming from the computer, then it's reasonable to say that the computer is exhibiting intelligence.

Create a script that plays the part of the independent computer, giving its user a simple medical diagnosis. The script should prompt the user with 'What is your problem?' When the user answers and presses *Enter*, the script should simply ignore the user's input, then prompt the user again with 'Have you had this problem before (yes or no)?' If the user enters 'yes', print 'Well, you have it again.' If the user answers 'no', print 'Well, you have it now.'

Would this conversation convince the user that the entity at the other end exhibited intelligent behavior? Why or why not?

3.7 (*Table of Squares and Cubes*) In Exercise 2.8, you wrote a script to calculate the squares and cubes of the numbers from 0 through 5, then printed the resulting values in table format. Reimplement your script using a for loop and the f-string capabilities you learned in this chapter to produce the following table with the numbers right aligned in each column.

number	square	cube
0	0	0
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125

3.8 (*Arithmetic, Smallest and Largest*) In Exercise 2.10, you wrote a script that input three integers, then displayed the sum, average, product, smallest and largest of those values. Reimplement your script with a loop that inputs four integers.

3.9 (*Separating the Digits in an Integer*) In Exercise 2.11, you wrote a script that separated a five-digit integer into its individual digits and displayed them. Reimplement your script to use a loop that in each iteration “picks off” one digit (left to right) using the // and % operators, then displays that digit.

3.10 (*7% Investment Return*) Reimplement Exercise 2.12 to use a loop that calculates and displays the amount of money you’ll have each year at the ends of years 1 through 30.

3.11 (*Miles Per Gallon*) Drivers are concerned with the mileage obtained by their automobiles. One driver has kept track of several tankfuls of gasoline by recording miles driven and gallons used for each tankful. Develop a sentinel-controlled-repetition script that prompts the user to input the miles driven and gallons used for each tankful. The script should calculate and display the miles per gallon obtained for each tankful. After processing all input information, the script should calculate and display the combined miles per gallon obtained for all tankfuls (that is, total miles driven divided by total gallons used).

```
Enter the gallons used (-1 to end): 12.8
Enter the miles driven: 287
The miles/gallon for this tank was 22.421875
Enter the gallons used (-1 to end): 10.3
Enter the miles driven: 200
The miles/gallon for this tank was 19.417475
Enter the gallons used (-1 to end): 5
Enter the miles driven: 120
The miles/gallon for this tank was 24.000000
Enter the gallons used (-1 to end): -1
The overall average miles/gallon was 21.601423
```

3.12 (*Palindromes*) A palindrome is a number, word or text phrase that reads the same backwards or forwards. For example, each of the following five-digit integers is a palindrome: 12321, 55555, 45554 and 11611. Write a script that reads in a five-digit integer and determines whether it’s a palindrome. [*Hint*: Use the // and % operators to separate the number into its digits.]

3.13 (*Factorials*) Factorial calculations are common in probability. The factorial of a nonnegative integer n is written $n!$ (pronounced “ n factorial”) and is defined as follows:

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1$$

for values of n greater than or equal to 1, with $0!$ defined to be 1. So,

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

which is 120. Factorials increase in size very rapidly. Write a script that inputs a nonnegative integer and computes and displays its factorial. Try your script on the integers 10, 20,

30 and even larger values. Did you find any integer input for which Python could not produce an integer factorial value?

3.14 (*Challenge: Approximating the Mathematical Constant π*) Write a script that computes the value of π from the following infinite series. Print a table that shows the value of π approximated by one term of this series, by two terms, by three terms, and so on. How many terms of this series do you have to use before you first get 3.14? 3.141? 3.1415? 3.14159?

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \cdots$$

3.15 (Challenge: Approximating the Mathematical Constant e) Write a script that estimates the value of the mathematical constant e by using the formula below. Your script can stop after summing 10 terms.

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$$

3.16 (*Nested Control Statements*) Use a loop to find the *two* largest values of 10 numbers entered.

3.17 (Nested Loops) Write a script that displays the following triangle patterns separately, one below the other. Separate each pattern from the next by one blank line. Use `for` loops to generate the patterns. Display all asterisks (*) with a single statement of the form

```
print('*', end='')
```

which causes the asterisks to display side by side. [*Hint:* For the last two patterns, begin each line with zero or more space characters.]

[illegible]

3.18 (*Challenge: Nested Looping*) Modify your script from Exercise 3.17 to display all four patterns side-by-side (as shown above) by making clever use of nested for loops. Separate each triangle from the next by three horizontal spaces. [*Hint: One for loop should control the row number. Its nested for loops should calculate from the row number the appropriate number of asterisks and spaces for each of the four patterns.*]

3.19 (Brute-Force Computing: Pythagorean Triples) A right triangle can have sides that are all integers. The set of three integer values for the sides of a right triangle is called a Pythagorean triple. These three sides must satisfy the relationship that the sum of the squares of two of the sides is equal to the square of the hypotenuse. Find all Pythagorean triples for side1, side2 and hypotenuse (such as 3, 4 and 5) all no larger than 20. Use a triple-nested for-loop that tries all possibilities. This is an example of “brute-force” computing. You’ll

learn in more advanced computer science courses that there are many interesting problems for which there is no known algorithmic approach other than sheer brute force.

3.20 (*Binary-to-Decimal Conversion*) Input an integer containing 0s and 1s (i.e., a “binary” integer) and display its decimal equivalent. The online appendix, “Number Systems,” discusses the binary number system. [Hint: Use the modulus and division operators to pick off the “binary” number’s digits one at a time from right to left. Just as in the decimal number system, where the rightmost digit has the positional value 1 and the next digit to the left has the positional value 10, then 100, then 1000, etc., in the binary number system, the rightmost digit has the positional value 1, the next digit to the left has the positional value 2, then 4, then 8, etc. Thus, the decimal number 234 can be interpreted as $2 * 100 + 3 * 10 + 4 * 1$. The decimal equivalent of binary 1101 is $1 * 8 + 1 * 4 + 0 * 2 + 1 * 1$.]

3.21 (*Calculate Change Using Fewest Number of Coins*) Write a script that inputs a purchase price of a dollar or less for an item. Assume the purchaser pays with a dollar bill. Determine the amount of change the cashier should give back to the purchaser. Display the change using the *fewest* number of pennies, nickels, dimes and quarters. For example, if the purchaser is due 73 cents in change, the script would output:

```
Your change is:
2 quarters
2 dimes
3 pennies
```

3.22 (*Optional else Clause of a Loop*) The while and for statements each have an optional else clause. In a while statement, the else clause executes when the condition becomes False. In a for statement, the else clause executes when there are no more items to process. If you break out of a while or for that has an else, the else part does *not* execute. Execute the following code to see that the else clause executes only if the break statement does not:

```
for i in range(2):
    value = int(input('Enter an integer (-1 to break): '))
    print('You entered:', value)

    if value == -1:
        break
else:
    print('The loop terminated without executing the break')
```

For more information on loop else clauses, see

<https://docs.python.org/3/tutorial/controlflow.html#break-and-continue-statements-and-else-clauses-on-loops>

3.23 (*Validating Indentation*) The file validate_indents.py in this chapter’s ch03-examples folder contains the following code with incorrect indentation:

```
grade = 93

if grade >= 90:
    print('A')
    print('Great Job!')
    print('Take a break from studying')
```

The Python Standard Library includes a code indentation validator module named **tabnanny**, which you can run as a script to check your code for proper indentation—this is one of many static code analysis tools. Execute the following command in the `ch03` folder to see the results of analyzing `validate_indents.py`:

```
python -m tabnanny validate_indents.py
```

Suppose you accidentally aligned the second `print` statement under the `i` in the `if` keyword. What kind of error would that be? Would you expect `tabnanny` to flag that as an error?

3.24 (Project: Using the *prospector* Static Code Analysis Tool) The *prospector* tool runs several popular static code analysis tools to check your Python code for common errors and to help you improve your code. Check that you’ve installed *prospector* (see the Before You Begin section that follows the Preface). Run *prospector* on each of the scripts in this chapter. To do so, open the folder containing the scripts in a Terminal (macOS/Linux), Command Prompt (Windows) or shell (Linux), then run the following command from that folder:

```
prospector --strictness veryhigh --doc-warnings
```

Study the output to see the kinds of issues *prospector* locates in Python code. In general, run *prospector* on all new code you create.

3.25 (Project: Using *prospector* to Analyze Open-Source Code on GitHub) Locate a Python open-source project on GitHub, download its source code and extract it into a folder on your system. Open that folder in a Terminal (macOS/Linux), Command Prompt (Windows) or shell (Linux), then run the following command from that folder:

```
prospector --strictness veryhigh --doc-warnings
```

Study the output to see more of the kinds of issues *prospector* locates in Python code.

3.26 (Research: *Anscombe’s Quartet*) In this book’s data science case studies, we’ll emphasize the importance of “getting to know your data.” The *basic descriptive statistics* that you’ve seen in this chapter’s and the previous chapter’s Intro to Data Science sections certainly help you know more about your data. One caution, though, is that different datasets can have identical or nearly identical descriptive statistics and yet the data can be significantly different. For an example of this phenomenon, research *Anscombe’s Quartet*. You should find four datasets and the associated visualizations. It’s the visualizations that convince you the datasets are quite different. In an exercise in a later chapter, you’ll create these visualizations.

3.27 (World Population Growth) World population has grown considerably over the centuries. Continued growth could eventually challenge the limits of breathable air, drinkable water, arable land and other limited resources. There’s evidence that growth has been slowing in recent years and that world population could peak some time this century, then start to decline.

For this exercise, research world population growth issues. This is a controversial topic, so be sure to investigate various viewpoints. Get estimates for the current world population and its growth rate. Write a script that calculates world population growth each year for the next 100 years, *using the simplifying assumption that the current growth rate will stay constant*. Print the results in a table. The first column should display the year

from 1 to 100. The second column should display the anticipated world population at the end of that year. The third column should display the numerical increase in the world population that would occur that year. Using your results, determine the years in which the population would be double and eventually quadruple what it is today.

3.28 (*Intro to Data Science: Mean, Median and Mode*) Calculate the mean, median and mode of the values 9, 11, 22, 34, 17, 22, 34, 22 and 40. Suppose the values included another 34. What problem might occur?

3.29 (*Intro to Data Science: Problem with the Median*) For an odd number of values, to get the median you simply arrange them in order and take the middle value. For an even number, you average the two middle values. What problem occurs if those two values are different?

3.30 (*Intro to Data Science: Outliers*) In statistics, outliers are values out of the ordinary and possibly way out of the ordinary. Sometimes, outliers are simply bad data. In the data science case studies, we'll see that outliers can distort results. Which of the three measures of central tendency we discussed—mean, median and mode—is most affected by outliers? Why? Which of these measures are not affected or least affected? Why?

3.31 (*Intro to Data Science: Categorical Data*) Mean, median and mode work well with numerical values. You can use them in calculations and arrange them in meaningful order. Categorical values are descriptive names like Boxer, Poodle, Collie, Beagle, Bulldog and Chihuahua. Normally, you don't use these in calculations nor associate an order with them. Which if any of the descriptive statistics are appropriate for categorical data?

